

git-archaeologist

Service cloud d'analyse de l'historique Git par LLM

Rapport de projet – DevOps & AWS

EPITA ING2 – 2025/2026

Maria Khvatova

<https://github.com/khvatovat/git-archaeologist>

Juin 2026

Table des matières

1	Presentation de l'application	2
1.1	Contexte	2
1.2	Stack applicative	2
2	Architecture AWS	2
2.1	Vue d'ensemble	2
2.2	Reseau (module <code>network</code>)	3
2.3	Sécurité réseau (module <code>security</code>)	3
2.4	Pourquoi DynamoDB et SQS	3
2.5	IAM – principe du moindre privilège	3
2.6	Secrets Manager	3
2.7	Vérification de signature webhook	4
3	Infrastructure as Code – Terraform	4
3.1	Structure modulaire	4
3.2	Backend d'état	4
3.3	Choix ECS Fargate	4
4	CI/CD – GitHub Actions	4
4.1	Pipeline A – Application (<code>app.yml</code>)	5
4.2	Pipeline B – Infrastructure (<code>terraform.yml</code>)	5
4.3	Pipeline C – Sécurité (<code>security.yml</code>)	5
4.4	Couverture des outils de sécurité	5
5	Automatisation – Ansible	6
6	Observabilité	6
6.1	Logs	6
6.2	Dashboard CloudWatch	6
6.3	Alarmes	6
7	Limites et améliorations possibles	7
8	Estimation des coûts (environnement dev)	7

1 Présentation de l'application

1.1 Contexte

git-archaeologist est un outil d'analyse de l'historique Git d'un fichier dans un depot GitHub. Pour chaque fichier analyse, il recupere les commits et pull requests associes, les regroupe par periode (appelées *eres*), génère un résumé narratif de chaque ère avec Claude Haiku, puis synthétise l'ensemble avec Claude Sonnet pour produire un rapport lisible sur l'évolution du fichier.

Ce projet transforme cet outil CLI en service cloud hébergé sur AWS :

- L'utilisateur enregistre un dépôt GitHub via une interface web statique (S3).
- GitHub envoie un webhook à chaque push ; l'API valide la signature HMAC-SHA256, crée une entrée dans DynamoDB et pousse un message dans SQS.
- Un worker consomme la file SQS, exécute le pipeline d'analyse et écrit le résultat dans DynamoDB.
- L'interface web permet de consulter les analyses par dépôt.

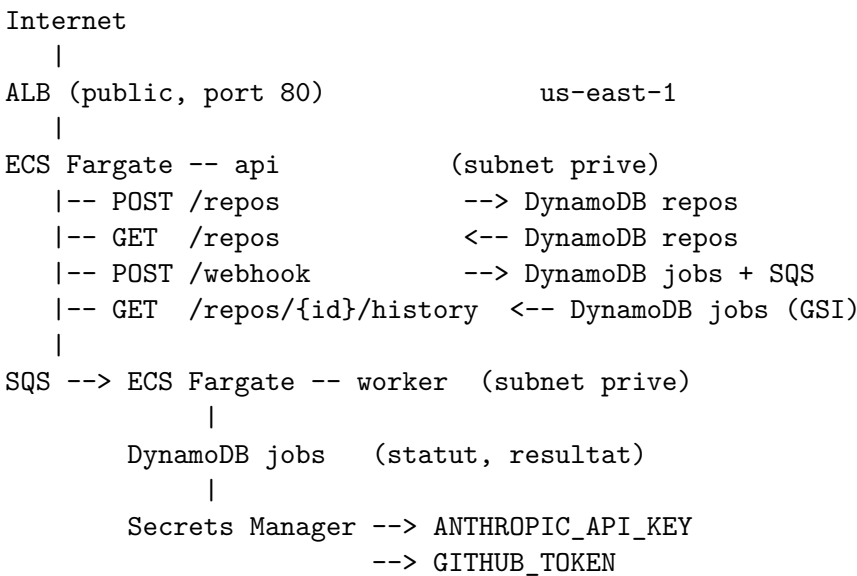
La logique d'analyse IA est réutilisée telle quelle depuis l'outil CLI. L'objectif du projet est l'infrastructure, l'automatisation, la sécurité et l'observabilité.

1.2 Stack applicative

Composant	Technologies
API	FastAPI (Python 3.11), Uvicorn, Pydantic
Worker	Python 3.11, boto3, Anthropic SDK
Frontend	HTML/JS statique hebergé sur S3
LLM	Claude Haiku (résumés d'eres), Claude Sonnet (synthèse)
Tests	pytest, moto (mock AWS), coverage

2 Architecture AWS

2.1 Vue d'ensemble



2.2 Réseau (module network)

Le VPC couvre deux zones de disponibilité. Chaque AZ contient un subnet public (ALB, NAT Gateway) et un subnet privé (tâches ECS). Les subnets privés n'ont pas de route vers l'Internet Gateway ; le trafic sortant passe par la NAT Gateway. Une seule NAT Gateway est provisionnée (optimisation de coût acceptable en développement ; en production, une par AZ garantit la résilience).

2.3 Sécurité réseau (module security)

Trois security groups strictement délimités :

Security group	Inbound	Outbound
alb-sg	0.0.0.0/0 :80	Tout
api-sg	alb-sg :8080 uniquement	Tout (APIs AWS via NAT)
worker-sg	Aucun	Tout

L'API n'est jamais accessible directement depuis internet, uniquement via l'ALB. Le worker n'a aucun port ouvert en entrée.

2.4 Pourquoi DynamoDB et SQS

DynamoDB est retenu car tous les accès sont par clé : recherche d'un job par `job_id`, liste des jobs d'un dépôt via GSI sur `repo_id+created_at`, enregistrement d'un dépôt par `repo_id`. Il n'y a pas de jointures ni d'agrégations. Le mode `PAY_PER_REQUEST` rend le coût négligeable en développement. RDS apporterait une complexité opérationnelle (backups, patches, connection pooling) sans aucun bénéfice pour ces patterns d'accès.

SQS est retenu car les jobs d'analyse sont longs (de l'ordre de la minute) et naturellement asynchrones. Le pattern producteur/consommateur est exactement le cas d'usage de SQS. Le *visibility timeout* et la dead-letter queue (3 tentatives) gèrent les échecs sans perte de données. Step Functions serait disproportionné pour un flux à une seule étape.

2.5 IAM – principe du moindre privilège

Chaque service n'a accès qu'aux ressources dont il a strictement besoin :

Rôle	Actions autorisées
api-task-role	DynamoDB PutItem/GetItem/Scan/Query (repos + jobs), SQS SendMessage
worker-task-role	DynamoDB GetItem/UpdateItem (jobs uniquement), SQS ReceiveMessage/DeleteMessage, Secrets Manager GetSecretValue
execution-role	ECR pull, CloudWatch Logs write, Secrets Manager GetSecretValue (injection dans le container)

Le worker ne peut ni lire ni écrire la table des dépôts. L'API ne peut ni lire les secrets ni accéder aux champs de mise à jour du worker. Les ARN des ressources sont explicitement spécifiés dans les politiques (pas de wildcard * sur `Resource`).

2.6 Secrets Manager

La clé Anthropic et le token GitHub sont stockés dans AWS Secrets Manager. Ils sont injectés dans le container worker via le champ `secrets` de la définition de tâche ECS, pas comme

variables d’environnement définies dans Terraform. Les valeurs n’apparaissent donc ni dans l’état Terraform, ni dans les logs, ni dans le code source.

2.7 Vérification de signature webhook

Chaque webhook GitHub entrant est vérifié par HMAC-SHA256 avec le secret par dépôt stocké dans DynamoDB. La comparaison utilise `hmac.compare_digest` pour prévenir les attaques temporelles. Les requêtes qui échouent la vérification retournent HTTP 401 avant toute écriture DynamoDB.

3 Infrastructure as Code – Terraform

3.1 Structure modulaire

L’infrastructure est organisée en 7 modules indépendants, câblés dans `terraform/envs/dev/main.tf` :

Module	Ressources créées
network	VPC, 2 subnets publics, 2 subnets privés, IGW, NAT Gateway, tables de routage
security	Security groups (ALB, API, worker), Secrets Manager (clé Anthropic, token GitHub)
messaging	File SQS + dead-letter queue (redrive après 3 tentatives)
data	Tables DynamoDB <code>repos</code> et <code>jobs</code> avec chiffrement SSE activé, index secondaires globaux
ecs	Cluster ECS Fargate, dépôts ECR (tags immuables), groupes de logs CloudWatch, rôles IAM, ALB, définitions de tâches et services
frontend	Bucket S3 (site statique), politique publique en lecture, upload des fichiers HTML
observability	Dashboard CloudWatch (5 widgets), 2 alarmes

3.2 Backend d’état

L’état Terraform est stocké dans un bucket S3 avec versioning activé. Le verrouillage est assuré par une table DynamoDB (`terraform-locks`), ce qui permet l’exécution du pipeline CI/CD sans risque de corruption de l’état.

3.3 Choix ECS Fargate

ECS Fargate a été retenu plutôt qu’EC2 ou Lambda :

- **EC2 + Auto Scaling** : nécessite de gérer les AMI, patches OS et placement d’instances. La charge ne justifie pas cette complexité.
- **Lambda** : la limite d’exécution à 15 minutes est incompatible avec les jobs d’analyse qui peuvent durer plusieurs minutes selon la taille du dépôt. La gestion des dépendances (Anthropic SDK, boto3) via Lambda layers serait aussi plus lourde.
- **ECS Fargate** : pas de gestion de serveurs, facturation à la seconde, isolation native des containers, intégration directe avec CloudWatch Logs, Secrets Manager et SQS.

4 CI/CD – GitHub Actions

Trois workflows indépendants sont mis en place, chacun avec un périmètre différent.

4.1 Pipeline A – Application (app.yml)

Déclenché sur push ou pull request vers `main`, uniquement si des fichiers sous `api/`, `worker/` ou des `*.py` à la racine ont changé.

1. **Unit Tests** : `pytest api/tests/ worker/tests/` avec `coverage`. Les dépendances AWS sont mockées avec `moto` (DynamoDB, SQS, Secrets Manager).
2. **SonarCloud** : analyse de qualité de code avec upload du rapport de couverture (push vers `main` uniquement).
3. **Trivy Filesystem Scan** : scan des vulnérabilités CRITICAL et HIGH sur le système de fichiers, résultats en SARIF.
4. **Build, Scan & Deploy** (push vers `main` uniquement) : build Docker API, scan Trivy de l'image, push ECR ; idem pour le worker ; puis `ecs update-service --force-new-deployment` sur les deux services.

4.2 Pipeline B – Infrastructure (terraform.yml)

Déclenché uniquement si des fichiers sous `terraform/` ont changé.

1. **Format & Validate** : `terraform fmt --check` et `terraform validate`.
2. **Checkov** : scan de sécurité IaC (règles Terraform), résultats en SARIF.
3. **Terraform Plan** (pull requests uniquement) : génère le plan et le publie en commentaire sur la PR pour revue manuelle avant merge.
4. **Terraform Apply** (push vers `main`) : `apply --auto-approve` avec variables sensibles injectées depuis les secrets GitHub.
5. **Ansible Post-Deploy** : après l'apply, un playbook attend que le health check de l'API réponde, synchronise les assets frontend vers S3, et soumet un job de test pour valider la chaîne complète.

4.3 Pipeline C – Sécurité (security.yml)

Déclenché sur chaque push vers `main`, chaque PR, et hebdomadairement (lundi 6h UTC).

1. **Semgrep SAST** : analyse statique avec les règles `p/python` et `p/owasp-top-ten`. Couvre Python, Terraform, Dockerfile et YAML. Bloquant sur les findings critiques.
2. **pip-audit** : scan des dépendances Python contre la base CVE, séparément pour l'API et le worker.
3. **TruffleHog** : scan de l'historique Git complet pour détecter des secrets committés par erreur. Mode `--only-verified` pour limiter les faux positifs.

4.4 Couverture des outils de sécurité

Outil	Pipeline	Périmètre
Semgrep	C	SAST Python + IaC, règles OWASP Top 10
Checkov	B	Mauvaises configurations Terraform (chiffrement, IAM, réseau)
Trivy	A	CVE sur filesystem et images Docker
TruffleHog	C	Détection de secrets dans l'historique Git
pip-audit	C	CVE dans les dépendances Python
SonarCloud	A	Qualité de code, dette technique, couverture de tests

5 Automatisation – Ansible

Ansible est utilisé en post-déploiement pour valider que l'infrastructure est opérationnelle et synchroniser les assets statiques. Le playbook (`ansible/deploy.yml`) s'exécute sur le runner GitHub Actions après `terraform apply` :

1. Récupère le DNS de l'ALB via `terraform output`.
2. Attend le passage du health check `GET /health` avec retry (10 tentatives, 15 secondes d'intervalle).
3. Synchronise les fichiers HTML du frontend vers le bucket S3.
4. Soumet un job de test via `POST /jobs` pour valider la chaîne API→SQS→worker.

Ce choix évite de dupliquer la logique de déploiement des assets dans Terraform et fournit une validation fonctionnelle automatisée après chaque déploiement.

6 Observabilité

6.1 Logs

Les logs de l'API et du worker sont streamés vers CloudWatch Logs via le driver `awslogs` configuré dans les définitions de tâches ECS (`/ecs/git-archaeologist-dev/api` et `/ecs/git-archaeologist-dev/worker`). Rétention fixée à 14 jours.

6.2 Dashboard CloudWatch

Un dashboard centralise cinq métriques clés :

Widget	Métrique
ALB Request Count	Nombre de requêtes par minute (SUM)
ALB Response Time p95	Latence au 95 ^e percentile
SQS Queue Depth	Messages visibles dans la file (backlog de jobs)
Worker Task Count	Tâches ECS en cours d'exécution (Container Insights)
ALB 5xx Error Count	Erreurs serveur par minute (SUM)

6.3 Alarmes

- **queue-depth-high** : file SQS > 20 messages pendant 2 minutes consécutives – le worker est bloqué ou sous-dimensionné.
- **api-5xx-high** : plus de 10 erreurs 5xx par minute pendant 2 minutes – problème applicatif ou infrastructure.

7 Limites et améliorations possibles

Limite actuelle	Mitigation en production
NAT Gateway unique	Une NAT GW par AZ pour éviter le point de défaillance sur la sortie privée
ALB en HTTP uniquement	Certificat ACM + listener HTTPS + redirection HTTP vers HTTPS
Worker non scalable	ECS Application Auto Scaling sur la métrique de profondeur de file SQS
Credentials AWS temporaires	OIDC federation GitHub→IAM (pas de secrets statiques à stocker)
Pas d'auth sur l'API	API Gateway + Cognito ou clé API; le secret webhook est la seule protection actuelle

8 Estimation des coûts (environnement dev)

Ressource	Coût mensuel estimé
ECS Fargate API (0,25 vCPU / 0,5 GB, 1 tâche)	\$6
ECS Fargate Worker (0,5 vCPU / 1 GB, 1 tâche)	\$11
NAT Gateway (1, trafic minimal)	\$35
ALB	\$16
Secrets Manager (2 secrets)	\$0,80
CloudWatch (logs + dashboard)	\$3
DynamoDB + SQS (trafic dev)	< \$1
Total	~\$73/mois

La NAT Gateway représente la majorité du coût. En production, des VPC Endpoints (S3, DynamoDB, SQS, Secrets Manager) réduiraient le trafic qui la traverse.